

Raport z projektu Klasyfikacja obrazów z Transfer Learning i ONNX Runtime

Przedmiot: Rozpoznawanie Obrazów

Temat: Transfer learning, eksport do ONNX, wdrożenie przez FastAPI i Docker

Model: ResNet18

Zbiór danych: Food-101

Spis treści

- [1. Cel projektu](#)
- [2. Opis zbioru danych](#)
- [3. Architektura modelu i transfer learning](#)
- [4. Trening](#)
- [5. Eksport do ONNX](#)
- [6. Benchmark: PyTorch i ONNX Runtime](#)
- [7. API i wdrożenie \(FastAPI i Docker\)](#)
- [8. Wnioski](#)
- [9. Instrukcja uruchomienia](#)

1. Cel projektu

Celem projektu było przeprowadzenie pełnego pipeline ML obejmującego:

- dobór gotowego modelu PyTorch i zastosowanie transfer learningu na nowym zbiorze danych,
- eksport wytrenowanego modelu do formatu ONNX,
- weryfikację zgodności predykcji między PyTorch a ONNX Runtime,
- pomiar i porównanie czasu inferencji obu środowisk uruchomieniowych,
- przygotowanie REST API (FastAPI) obsługującego predykcje na przesłanych obrazach,
- konteneryzację aplikacji z użyciem Dockera.

2. Opis zbioru danych

Food-101 to publicznie dostępny zbiór danych zawierający zdjęcia potraw z 101 różnych kategorii.

Cecha	Wartość
Liczba klas	101
Liczba obrazów (train)	75 750

Cecha	Wartość
Liczba obrazów (test)	25 250
Łącznie	101 000
Rozmiar na dysku	~5 GB
Źródło	kaggle

Przykładowe klasy: `pizza`, `ramen`, `tiramisu`, `sushi`, `hot_dog`, `ice_cream`.

ResNet18 trenowany na ImageNet rozpoznaje ogólne kategorie obiektów. ImageNet zawiera kilka kategorii jedzenia, jednak Food-101 definiuje 101 szczegółowych klas, na których ResNet18 nie był trenowany.

3. Architektura modelu i transfer learning

Model bazowy

Zastosowano **ResNet18** - sieć składająca się z 18 warstw i około 11 milionów parametrów, pretrenowaną na zbiorze **ImageNet** (1000 klas, 1.2 mln obrazów).

Transfer learning

Transfer learning polega na ponownym wykorzystaniu wiedzy zdobytej przez model podczas treningu na dużym zbiorze danych i zaadaptowaniu jej do nowego zadania klasyfikacji. Warstwy konwolucyjne ResNet18 nauczyły się wykrywać uniwersalne cechy wizualne - krawędzie, tekstury, kształty - które są przydatne niezależnie od konkretnego zadania klasyfikacji.

W projekcie zastosowano podejście fine-tuning: wszystkie wagi modelu pozostały odblokowane i były aktualizowane podczas treningu, jednak inicjalizowane wartościami z ImageNet. Jedyna modyfikacja architekuralna polegała na zastąpieniu ostatniej warstwy `FC` nową warstwą liniową z 101 wyjściami odpowiadającymi klasom Food-101. Oryginalna warstwa miała 1000 wyjść odpowiadających klasom ImageNet.

Wybór ResNet18 był podyktowany jego niewielkim rozmiarem (szybki trening i inferencja), dobrą jakością nauczonej reprezentacji cech oraz szerokim wsparciem w ekosystemie PyTorch i ONNX.

Preprocessing obrazów

Wszystkie obrazy były skalowane do rozmiaru 224×224 pikseli, konwertowane do tensora i normalizowane przy użyciu średnich i odchyłeń standardowych obliczonych dla zbioru ImageNet: `mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`.

Dla zbioru treningowego zastosowano dodatkowo augmentację w postaci losowego odbicia poziomego `RandomHorizontalFlip`, co zwiększa różnorodność danych treningowych i pomaga ograniczyć przeuczenie.

4. Trening

Hiperparametry

Parametr	Wartość
Optimizer	Adam
Learning rate	0.0001
Batch size	64
Liczba epok	10
Funkcja straty	CrossEntropyLoss
num_workers	16
pin_memory	True

Wybór learning rate: Wartość `0.0001` jest typowa dla transfer learningu. Większe wartości mogłyby zdestabilizować już nauczone wagi konwolucyjne, mniejsze powodowałyby zbyt wolną konwergencję.

Wyniki treningu

Epoka	Dokładność walidacyjna
1	70.4%
2	75.4%
3	75.9%
4	75.5%
5	76.6%
6	76.8%
7	75.9%
8	76.2%
9	75.1%
10	76.0%

Analiza wyników

Model osiągnął dokładność walidacyjną na poziomie **~76%** po 10 epokach treningu. Wynik jest zadowalający biorąc pod uwagę:

- krótki trening
- brak rozbudowanej augmentacji danych

- zastosowanie lekkiego modelu

Widoczna jest lekka oscylacja dokładności między epokami (75.1%–76.8%), co sugeruje że model nie uległ przeregulowaniu. Dalsze poprawy byłyby możliwe przez:

- zwiększenie liczby epok
- zastosowanie learning rate schedulera (np. `CosineAnnealingLR`)
- rozbudowanie augmentacji
- użycie większego modelu

5. Eksport do ONNX

Model zapisany jako `model.pth` został wyeksportowany do formatu **ONNX** przy użyciu `torch.onnx.export`.

Parametry eksportu

```
torch.onnx.export(  
    model, dummy_input, 'model.onnx',  
    input_names=['input'],  
    output_names=['output'],  
    dynamic_axes={'input': {0: 'batch_size'}, 'output': {0: 'batch_size'}},  
    opset_version=18  
)
```

Użyto `dynamic_axes` aby model ONNX obsługiwał zmienną wielkość batch.

Weryfikacja zgodności predykcji

Zgodność predykcji obu środowisk została zweryfikowana skryptem `verify.py`, który porównuje wyniki dla identycznego wejścia (losowy tensor $1 \times 3 \times 224 \times 224$). Wynik weryfikacji:

```
PyTorch class: 26  
ONNX Runtime class: 26  
Max diff: 0.000005  
Compatibility: OK
```

Oba modele zwróciły identyczną klasę (26). Maksymalna różnica surowych wartości wyjściowych wyniosła `0.000005`, co mieści się w granicach błędu numerycznego.

6. Benchmark: PyTorch i ONNX Runtime

Benchmark przeprowadzono na **CPU** (100 przebiegów, obraz $1 \times 3 \times 224 \times 224$, uśredniony czas).

Runtime	Czas inferencji	Przyspieszenie
PyTorch CPU	184.25 ms/obraz	1x
ONNX Runtime CPU	4.37 ms/obraz	42.15x

Analiza wyników benchmarku

ONNX Runtime jest **42-krotnie szybszy** od PyTorch na CPU. Wynika to z kilku czynników:

- **Optymalizacja grafu**- ONNX Runtime przeprowadza wiele optymalizacji na grafie obliczeniowym
- **Backend wykonawczy** - korzysta z zoptymalizowanych kerneli
- **Brak overhead PyTorch** - PyTorch na CPU niesie ze sobą narzut interpretera i dynamicznego grafu obliczeniowego

Praktyczne znaczenie: W scenariuszu wdrożenia produkcyjnego bez GPU, ONNX Runtime pozwala obsłużyć 42x więcej zapytań na tej samej infrastrukturze w porównaniu do natywnego PyTorch.

7. API i wdrożenie (FastAPI i Docker)

FastAPI

Aplikacja udostępnia dwa endpointy:

Endpoint	Metoda	Opis
<code>/health</code>	GET	Sprawdzenie dostępności serwisu
<code>/predict</code>	POST	Predykcja klasy dla przesłanego obrazu

Inferencja odbywa się przez **ONNX Runtime**, co zapewnia szybką odpowiedź bez GPU.

Format odpowiedzi `/predict`:

```
{
  "class_id": 72,
  "confidence": 0.9341
}
```

Docker

Aplikacja została skonteneryzowana z użyciem Dockera. Kontener zawiera:

- środowisko Python 3.11-slim,
- zainstalowane zależności (`onnxruntime`, `fastapi`, `uvicorn`, `pillow`),
- plik modelu `model.onnx`.

Model ONNX jest montowany jako wolumen (`./model:/app/model`).

8. Wnioski

- Transfer learning na ResNet18 pozwolił osiągnąć **~76% dokładności** na Food-101 w 10 epokach treningu, co potwierdza skuteczność tej techniki przy ograniczonym czasie obliczeniowym.
- Eksport do ONNX jest bezstratny - predykcje obu środowisk są identyczne.

- ONNX Runtime oferuje **42-krotne przyspieszenie** inferencji na CPU względem PyTorch, co ma istotne znaczenie w scenariuszach produkcyjnych bez dostępu do GPU.
- FastAPI i Docker tworzą przenośną, łatwą do wdrożenia aplikację gotową do uruchomienia na dowolnej platformie.

9. Instrukcja uruchomienia

Wymagania

- Python 3.11+
- Docker i Docker Compose

Instalacja zależności

```
pip install -r requirements.txt
```

Trening modelu

```
python src/train.py
```

Pobiera Food-101 automatycznie (~5 GB). Zapisuje `model/model.pth`.

Eksport do ONNX

```
python src/export_onnx.py
```

Zapisuje `model/model.onnx`.

Benchmark

```
python src/benchmark.py
```

Uruchomienie API przez Docker

```
docker-compose up -d
```

API dostępne pod adresem `http://localhost:3333`.

Testowanie API

```
# Health check
curl http://localhost:3333/health

# Predykcja
curl -X POST http://localhost:3333/predict \
  -F "file=@obraz.jpg"
```

Zatrzymanie kontenera

```
docker compose down
```